



Trabajo Práctico Final Visión Por Computadora

Materia: **IA5.1 Visión Por Computadora**

Carrera: **Tecnicatura Universitaria en Inteligencia Artificial**

Alumno: **Gonzalo Uriel Calvo Rodriguez : C-7345/8**

Profesores: **Juan Pablo Manson, Brugé Lucas, Ferrucci Constantino.**

**Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura**

Introducción.....	3
ETAPA 1.....	4
Resumen.....	4
resolución.....	4
ETAPA 2.....	7
Resumen.....	7
resolución.....	7
ETAPA 3.....	14
Resumen.....	14
resolución.....	15
ETAPA 4.....	17
Resumen.....	17
Resolución.....	18
Métricas.....	18
Optimización.....	24
CONCLUSIÓN.....	25

Introducción

En este trabajo práctico se nos solicita crear un pipeline de visión por computadora completo que identifique razas de perros de diversas imágenes. El trabajo está dividido en 4 etapas principales donde se van tratando diversos temas y formas de afrontar la identificación de razas.

Partimos de un dataset de Kaggle donde hay 9346 imágenes ya divididas en 7846 de entrenamiento, 700 de validación y 700 de test.

En la primer etapa la idea principal es utilizar un modelo pre-entrenado en ImageNet para extraer embeddings de cada imagen del dataset, para luego a través de una base de datos vectorial poder hacer una búsqueda por similitud, a su vez también se crea una sencilla interface en la aplicación Gradio para poder visualizar estas búsquedas por similitud.

En la siguiente etapa y para mí la más divertida, tenemos que entrenar nuestros propios modelos de clasificación. Podemos elegir hacer un fine-tuning a un modelo ya entrenado o hacer un modelo de clasificación convolucional propio desde cero.

Luego probarlo nuevamente en la aplicación de Gradio (un poco modificada).

Ya superando la etapa 2 llegamos a la 3er etapa de este trabajo práctico, donde ahora utilizamos YOLO (pre-entrenado) para detectar la presencia de perros en una imagen.

Para luego que el usuario pueda pasarle una imagen compleja y el modelo pueda encontrar un perro en ella y extraer su bounding box. Dicho bounding box posteriormente hay que mandárselo al “mejor modelo de la etapa 2” CNN-Custom o Fine-Tuning.

En la última etapa tenemos que armar un mini-dataset de 10 imágenes “complejas” con perros y otros objetos en las mismas, a estas, de manera manual, se les hace un bounding box y se les asigna un label/tag con la raza correspondiente al perro de la imagen.

Luego de esto cargamos el nuevo dataset y sus “anotaciones” con coordenadas de los bounding boxes y razas para cada imagen y procedemos a evaluar el pipeline creado en la etapa anterior (Yolo + mejor modelo etapa 2) midiendo el mismo con (mAP, IoU, Precision, Recall y F1-Score).

Siendo con la etapa 4 procedemos a optimizar el modelo, tenemos 2 maneras “cuantización” o “exportar a formato optimizado”.

Y como último crear un script en python que utilice nuestro pipeline completo (detector + clasificador) para generar anotaciones de forma automática.

ETAPA 1

Resumen

Se nos solicita utilizar un modelo pre-entrenado en ImageNet para extraer embeddings, crear una base de datos vectorial con los mismo, desarrollar una interface simple en gradio para poder subir una imagen, extraer su embedding y hacer una búsqueda de las 10 imágenes más similares dentro de nuestra base vectorial, luego de esto implementar una lógica que prediga la raza del perro de la imagen de entrada.

Cómo último con un conjunto de prueba de 5 o 10 imágenes por raza (distinto al de busqueda) calculara la métrica NDCG@10.

resolución

Para empezar tenía que elegir un modelo pre-entrenado en ImageNet por ejemplo ResNet50, el cual como se daba de ejemplo fue el que decidí probar primero y nunca cambié.

Opte por Pytorch como framework por su flexibilidad para manipular modelos pre-entrenados y su compatibilidad con GPU, lo que aceleraría etapas posteriores (como el Fine-Tuning o la inferencia en grandes volúmenes de datos)

Buscando el modelo en la página de pytorch veo que tiene distintos tipos de pesos, por defecto no utiliza ninguno.

Inicialmente cargué el modelo sin pesos (weights=None), pero al evaluar su salida, noté que el tensor final correspondía a las probabilidades de clasificación, no a características genéricas.

Esto lo solucioné “cortando” la última capa del modelo y quedándome con toda la parte convolucional del Resnet50, es decir la capa inicial (conv1, bn1, relu, maxpool) y los 4 bloques de layers (layer1, layer2, layer3, layer4).

Esto sirve cuando queremos usar ResNet50 como extractor de características, donde la salida es un tensor con las características (de 2048 dimensiones) de la imagen.

Lo siguiente es indexar estos vectores de salida a una base de datos vectorial para permitir búsquedas eficientes.

Para indexar y buscar los embeddings de manera eficiente, seleccioné ChromaDB ya que fue recomendado en la primera clase consulta y recomendación de compañeros de clase que ya lo habían utilizado anteriormente y en lo personal no me dio problemas para probar otras opciones.

Para la indexación lo primero que hice fue extraer todos los embeddings de las imágenes del dataset (solo train) usando mi ResNet50 modificado. Almacené los vectores con ChromaDB, asociando cada uno con su metadato, ruta de la imagen y raza. Para la Búsqueda por Similitud utilicé distancia coseno, que compara el embedding de la imagen de entrada con los vectores indexados.

Una vez indexados los vectores en ChromaDB, con la aplicación gradio utilizamos esta base de datos para realizar búsquedas en tiempo real cuando el usuario sube una imagen.

Gradio

Llegando a la interfaz Gradio, la verdad es que es muy sencilla, donde tiene como parámetros una función central, que recibe una imagen (subida por el usuario) extrae su embedding y consulta en la base vectorial para encontrar las 10 imágenes más similares.

Un input, es lo que permite al usuario subir una imagen, tiene un parámetro adicional para que la imagen se cargue en formato “PIL” y el output son 11 imágenes, la original y las 10 más similares ordenadas por similitud de mayor a menor.

A partir de estos 10 resultados obtenidos se implementó una lógica para predecir la raza del perro de la imagen de entrada, elegí el “voto mayoritario” ya que me pareció la más obvia y no se me ocurrió una idea mejor o más eficiente para esta tarea.

Evaluación del Sistema con NDCG@10

El NDCG@10 evalúa la calidad del sistema de búsqueda por similitud teniendo en cuenta dos puntos claves.

La relevancia de los resultados, es decir cuántas de las 10 razas pertenecen a la misma raza que la imagen de consulta, y la posición de los resultados relevantes, ya que los primeros puestos tienen más peso que los últimos.

Para calcular esta métrica utilizo SkLearn en específico `ndcg_score`, que toma de entrada 2 valores `y_true`, `y_score`.

Empiezo cargando el dataset de test, para que no haya ningún tipo de sesgo, de cada imagen obtengo a que clase (raza) pertenece, obtengo su embedding y realizo la búsqueda por similitud.

Ahora calculo la relevancia o `y_true` (es una lista binaria donde 1 es que la imagen “recuperada” de ChromaDB es de la misma raza que la consulta, y 0 lo contrario).

Para la puntuación o `y_score` son las distancias devueltas en la consulta a ChromaDB, NDCG interpreta que los valores más altos son los más relevantes, mientras que ChromaDB interpreta valores bajos = más similares, por ese motivo se invierten las distancias.

Ahora que ya tenemos nuestro `y_true` e `y_score`, se lo damos como entrada a `ndcg_score` y este nos devuelve una puntuación final promedio para todas las imágenes de prueba.

Los valores que puede tomar `ndcg_scores` van de 0 (peor) a 1 (mejor).

Antes de hablar sobre el valor obtenido de NDCG quería mencionar un punto clave a mi parecer para obtener un valor alto, y es que mirando las queries de las búsquedas por similitud observé que las distancias tenían valores “muy altos” en comparación a ejemplos vistos en la documentación o ejemplos de internet. Ante lo cual pruebo con cambiar los pesos que mencioné anteriormente del modelo ResNet50 (`weights=None`) por (`weights=IMAGENET1K_V2`), los mismos pesos fueron sacados de la página de pytorch resnet50 donde dicen que esta nueva ponderación mejoran los resultados. Al probar esta nueva ponderación veo que las distancias pasaron de 200 o 300 a 30,40,50,etc. Valores mucho más chicos y precisos, haciendo que el NDCG aumente casi un 0.10 con respecto a no usar pesos.

Una vez ejecutado obtenemos que un `NDCG@10 (sklearn): 0.9621302859219101`

Lo que significa que en promedio, las imágenes relevantes (de la misma raza que la consulta) aparecen en las primeras posiciones de los 10 resultados de la query.

Por ejemplo, si buscamos un Bulldog, es muy probable que los primeros perros 3 o 5 perros sean Bulldogs y el resto sean razas similares.

La alta puntuación da a entender que los embeddings extraídos por ResNet50 captura características muy buenas para cada raza de perro y que la base de datos vectorial ChromaDB recupera con éxito los vectores de características más similares.

También se puede pensar que la posibilidad de obtener un valor tan alto puede ser porque las imágenes de una misma raza podrían ser muy similares haciendo que la distancia entre embeddings sea más estrecha.

O que las razas del dataset son muy distintas unas entre otras, facilitando la diferenciación

ETAPA 2

Resumen

En esta etapa se plantea entrenar nuestros propios modelos para mejorar la representación de características y la clasificación. Se nos da a elegir entre 2 maneras de hacer esto, una entrenando un modelo ResNet18 haciéndole un Fine-Tuning sobre el dataset de 70 razas de perros, y la otra opción es diseñar y entrenar un modelo de Clasificación Convolutiva (CNN) propio desde cero.

Luego de esto a estos modelos explicar sus métricas (Sensibilidad, Especificidad, Precisión, Exactitud y puntaje F1).

Una vez terminado el entrenamiento, actualizar la interfaz de la aplicación Gradio, para que ahora los embeddings se puedan realizar tanto con el ResNet18 como con el modelo custom y realizar la búsqueda por similitud.

resolución

Fine-Tuning

Bien, el primer paso en esta etapa fue entender qué es el *fine-tuning* y cómo aplicarlo bien. Básicamente, consiste en adaptar un modelo pre-entrenado a una tarea nueva (en este caso, clasificar 70 razas de perros).

Para eso, modifiqué la arquitectura del modelo, cambiando la última capa *fully connected* (FC) por una que tenga la misma cantidad de salidas que clases, ya que originalmente está pensada para 1000 clases genéricas.

Dentro de esta modificación hice una capa básica de 256 neuronas, una relu y un dropout para evitar overfitting.

Luego, entrené esta nueva capa. Pero además, para hacer un entrenamiento más completo, también descongelé las capas convolucionales del modelo. Esto permite ajustar todo el modelo a mi dataset, aunque es más costoso computacionalmente. Si no hiciera esto, solo entrenaría la FC, y el aprendizaje sería superficial.

Entrenamiento

Una vez tenemos listo el modelo para entrenar, lo pasamos a GPU (en mi caso tengo la posibilidad de acelerar el entrenamiento desde este lado). Configuro todo lo necesario para el entrenamiento como la función de pérdida, usando CrossEntropyLoss.

Para la optimización de pesos usó Adam, con un Learning_Rate de 0.001 y un weight decay de $1e-4$ para evitar overfitting. También utilizo un scheduler (ReduceLROnPlateau) que reduce el Learning Rate a la mitad si la precisión en validación no mejora después de 5 etapas seguidas.

Para entrenar el modelo usamos el Dataset de entrenamiento con data augmentatio, donde se aplican transformaciones aleatorias a la imágenes para que el modelo aprenda mejor, dentro de las transformaciones que uso están, espejado horizontal, rotacione de hasta 15 grados, cambios leves en brillo, contraste y saturación, también se hace un resize a 224x224 (aunque las imágenes ya estaban en esas dimensiones). Después, cada imagen se convierte en un tensor y se normaliza con los valores estándar de ImageNet.

El entrenamiento esta seteado para correr 300 epocas com maximo, pero uso un Early Stopping con una paciencia de 20, esto quiere decir que si durante 20 epocas seguidas la precisión en validación no mejora, el entrenamiento se detiene automáticamente, y se recupera la mejor época obtenida.

En cada época que el modelo se entrena, se calcula la pérdida y la precisión. Después pasa a evaluación, donde calcula lo mismo, pero con el dataset de validación. Al final, se carga el modelo con los mejores pesos obtenidos durante el entrenamiento.

En la pérdida mencionada, medimos que tan mal están las predicciones del modelo en comparación con las etiquetas reales, cuanto menor sea, mejor está aprendiendo el modelo. La precisión es el porcentaje de predicciones correctas sobre el total, es una métrica más que nada intuitiva, donde podemos observar que si tenemos un 60% o 0.60, quiere decir que 6 de cada 10 predicciones fueron correctas.

Una vez finalizado el entrenamiento, evaluamos el rendimiento final del mejor modelo en el dataset test. Para esto importo SkLearn Classification_report, la cual nos devuelve varias métricas por clase.

Precisión: que como ya mencione, es todas las veces que el modelo predijo una clase, acertó la misma.

Recall: es de todas las veces que debería haber predicho una clase, cuantas veces lo hizo.

F1-Score: es una combinación de precisión y recall (un promedio).

Support: es la cantidad de muestras por clase.

En general, los resultados obtenidos al hacer Fine-Tuning fueron muy buenos.

Obteniendo un Accuracy global del modelo de 0.95. El F1-score promedio tambien es de 0.96, lo que indica un balance entre precisión y recall en casi todas las clases, en algunas clases tuvieron valores más bajos, que probablemente se deba a razas visualmente similares o poca variabilidad en el dataset.

Acá me gustaría hablar sobre el Recall que me costó un rato entender su diferencia con la precisión y me gustaría nombrarla.

Ejemplo con "Beagles"

Si tenemos 10 Beagles reales y el modelo encuentra 8 el recall es $8/10 = 80\%$

Mientras que si el modelo predice 10 beagles, pero solo había 6 de verdad la precisión es $6/10 = 60\%$.

Recall se mueve sobre los valores reales, mientras que precisión sobre los predichos.

Además de estas métricas también calculó la especificidad de cada clase, donde medimos la capacidad del modelo para evitar falsos positivos, es decir, de todos los casos que no pertenecen a una clase, cuántos fueron correctamente clasificados como "no esa clase".

En pocas palabras, nos sirve para saber que tan bien el modelo descarta las clases incorrectas.

Para calcularlo usamos la Matriz de confusión, por cada clase se toma:

TP (verdadero positivo): predichos correctamente como esa clase.

TN (verdadero negativo): predichos correctamente como no esa clase.

FN (falso positivo): predichos como esa clase pero no lo son.

FN (falsos negativos): deberían haber sido esa clase, pero no lo fueron

La fórmula para calcular la especificidad es **especificidad** = $TN / (TN + FP)$

El valor promedio obtenido fue de 0.999, lo que indica que el modelo así nunca confunde al asignar clases incorrectas, es decir, si algo no es de una determinada raza, el modelo lo clasifica bien como "otra cosa"

Custom CNN

Me pareció bien hacer también el Custom CNN diseñado de cero para clasificar 70 raza de perros, por eso lo tomé un poco como un desafío propio ver podía hacer.

A la hora de crear la red, lo primero que defino es el bloque de extracción de características o self.features, esta es la parte encargada de procesar la imagen y extraer patrones visuales (bordes, texturas, formas, etc.)

Luego las capas convolucionales(`nn.conv2d`), que aplican filtros (kernels) que extraen características. Esto toma como entrada el canal de la imagen en este caso como es RGB son 3, el número de filtros, el tamaño del kernel y `padding = 1` que añade un borde de ceros alrededor de la imagen para evitar que se reduzca demasiado la imagen.

Luego tenemos Batch Normalization(`nn.BatchNorm2d`), que normaliza las salidas de cada capa para acelerar el entrenamiento y evitar inestabilidad, también reduce la necesidad de ajustar mucho el Learning Rate.

Aplicamos ReLU(`nn.ReLU`) que introduce no-linealidad para valores positivos, es simple, eficiente y evita problema del “gradiente desaparecido”. Este último simplemente resumido, cuando se calcula la pérdida en el entrenamiento de la red, este se propaga hacia atrás para ajustar los pesos, este ajuste se hace usando gradiente. Pero en algunas funciones a medida que avanzan las capas se vuelven chicos, tan chicos que llegan casi a cero, y esto hace que las capas del principio dejen de aprender.

Max Pooling (`nn.MaxPool2d`), reduce el tamaño de la imagen, conservando las características más importantes, usó el parámetro `Stride=2` que divide el ancho y alto por 2 en cada paso, esto hace el modelo más rápido y menos propenso a overfitting.

`AdaptiveAvgPool2d` reduce la imagen en un tamaño fijo (1x1) al final, independientemente de la entrada.

Luego en el bloque clasificador (`self.classifier`), se toman las características extraídas y se decide a qué raza pertenece la imagen.

`Flatten(nn.Flatten)`, convierte los mapas de características 2d en un vector 1d para la capa FC o fully connected.

Capas Fully Connected(`nn.Linear`) conecta todas las neuronas para aprender a relacionar complejas entre características. Uso una estructura de 1024 - 512 - 256 - 70 (numero de salida 70 razas), combina todos los patrones detectados para tomar una decisión final.

`Dropout(nn.Dropout)`, “apaga” aleatoriamente neuronas durante el entrenamiento para evitar overfitting, con `p=0.4` (el 40% de neuronas desactivadas en la primera capa).

Resumiendo la CNN personalizada sigue la siguiente estructura en primera instancia se extrañen características: bloques convolucionales con BatchNorm y ReLU, que gradualmente reducen la resolución espacial, pero aumentan la profundidad de 32 a 1024 canales. el Padding mantiene la información en bordes, y el max-polling conserva solo las características más relevantes.

Clasificación, se usa capas FC con Dropout de 40% y 30% para evitar el overfitting, y convirtiendo las características en una predicción final entre las 70 razas.

El entrenamiento de esta red neuronal es exactamente el mismo que el de Fine-Tuning, utilizando los mismo parámetros para todo.

Métricas obtenidas

Una vez que finaliza el entrenamiento del modelo *custom CNN*, evaluamos su rendimiento con el conjunto de test. Para esto, usamos tres métricas por clase: **precisión**, **recall** y **f1-score**, junto con el **accuracy** global, los promedios (macro y weighted) y la **especificidad media**.

```
print(métricas(modelo_custom))
```

✓ 5.6s				
	precision	recall	f1-score	support
0	0.90	0.90	0.90	10
1	1.00	0.90	0.95	10
2	0.82	0.90	0.86	10
3	0.82	0.90	0.86	10
4	0.78	0.70	0.74	10
5	0.77	1.00	0.87	10
6	0.91	1.00	0.95	10
7	1.00	0.90	0.95	10
8	1.00	0.90	0.95	10
9	0.91	1.00	0.95	10
10	1.00	0.90	0.95	10
11	0.82	0.90	0.86	10
12	0.91	1.00	0.95	10
13	0.89	0.80	0.84	10
14	0.90	0.90	0.90	10
15	0.89	0.80	0.84	10
16	0.62	0.80	0.70	10
17	0.91	1.00	0.95	10
18	0.89	0.80	0.84	10
19	0.78	0.70	0.74	10
20	0.78	0.70	0.74	10
21	0.73	0.80	0.76	10
22	0.83	0.50	0.62	10
...				
accuracy			0.86	700
macro avg	0.87	0.86	0.86	700
weighted avg	0.87	0.86	0.86	700

Podemos observar un Accuracy del 86% podemos decir que el modelo clasificó correctamente 86 de cada 100 imágenes, lo considero un buen valor teniendo en cuenta que no realizo transfer learning, entreno con un dataset mucho más chico que un modelo como ResNet18 y que tenemos para clasificar 70 clases distintas, y que cada clase tiene apenas 10 imágenes en el dataset de test, haciendo que cada error tengo un peso alto en la métrica.

La precisión promedio (macro y weighted): 0.87 mide cuantas predicciones del modelo fueron correctas, que sea alta nos indica que cuando el modelo dice que una imagen es de una raza, suele tener razón.

El recall promedio es de 0.86, el mismo mide cuantas veces el modelo logró detectar correctamente una raza cuando realmente aparecía. Si el recall fuera bajo, indicaría que el modelo está “dejando pasar” muchos casos correctos.

F1-score promedio de 0.86 es una combinación de las dos anteriores, tener un f1-score promedio alto es un indicio de equilibrio entre acertar y detectar correctamente.

Aunque hay clases con un f1-score promedio de 1.00, también hay varias con valores por debajo de 0.75 como la clase 22 (Bulldog): f1-score 0.62 (precisión 0.83, recall 0.50), el modelo predice bien cuando acierta, pero se le escapan muchos casos reales, probablemente no esté logrando generalizar bien esa raza.

La clase 27 (Clumber): f1-score 0.59 (precisión 0.71, recall 0.50), un bajo recall indica que está reconociendo solo la mitad de los casos reales.

Y la clase 33 (Dalmation): f1-score 0.61 (precisión 0.54, recall 0.70), en este caso el modelo encuentra muchos casos reales, pero se equivoca bastante cuando predice. interpreto que el modelo está siendo “confiado” y predice clases erróneas muchas veces, pero de esas predicciones, casi la mitad son incorrectas. Probablemente está confundiendo otra clase con esta.

Y por último se calculó la especificidad que da 0.9980, esta métrica mide qué tan bien el modelo evita confundir una clase con otras, este valor estando alto indica que el modelo casi nunca asigna incorrectamente una clase cuando no corresponde, es decir, si una imagen no es de la raza 1 (Afghan), por ejemplo, el modelo rara vez lo clasifica como tal. Este resultado nos muestra que el modelo no no adivina al azar ni etiqueta equivocadamente, comete errores, pero cuando lo hace, pareciera ser por razas similares o difíciles de clasificar, no de forma descontrolada.

Complicaciones de esta primer parte de la etapa

La primer complicación se dió desde un punto de vista mía en la comprensión de la consigna sobre el fine-tuning, como él mismo no lo encontré en el material de clase, pensé que había que realizar un transfer learning, del punto de vista que tomaba del ResNet18 solamente la manera en que hace embeddings y yo entrenar el clasificador de cero con sus pesos congelados. Este error por suerte se me solucionó rápido ya que aunque había hecho el código y entrenado el modelo, en la primera clase de consulta, me di cuenta de mi error.

Otro problema cómico en el cual me perdí algunas horas sin entender a qué se debía fue que a la hora de cargar el dataloader de validación, por un error de copiar y pegar mi propio código, estaba cargando siempre train. provocando que los entrenamientos den por demás de bien, ya que estaba entrenando y validando sobre los mismo datos todo el tiempo.

Luego de esto la verdad que con el fine-tuning no tuve problemas.

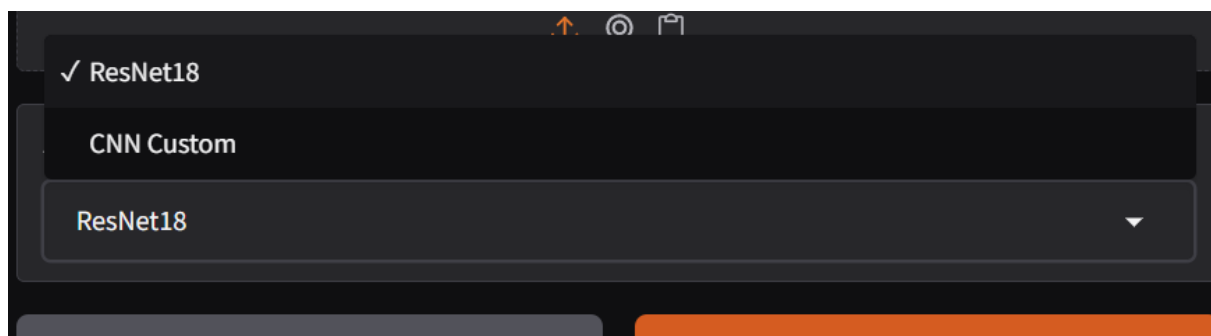
Pasando al custom cnn, tuve diversos problemas para lograr que mejore su precisión a la hora de validar en el entrenamiento. La primera solución fue subir el Learning Rate de 0.0001 a 0.001, haciendo que aprenda más rápido el modelo, la segunda y considero que una de las más importantes mejoras es aplicar data augmentation al dataset de train, surtiendo una mejora notable en el paso de las épocas y pasando de un rendimiento máximo de 20% o 25% de precisión por época a un 35% o 45%. y el cambio indudablemente más importante hecho ahora a la red neuronal fue agregarle más

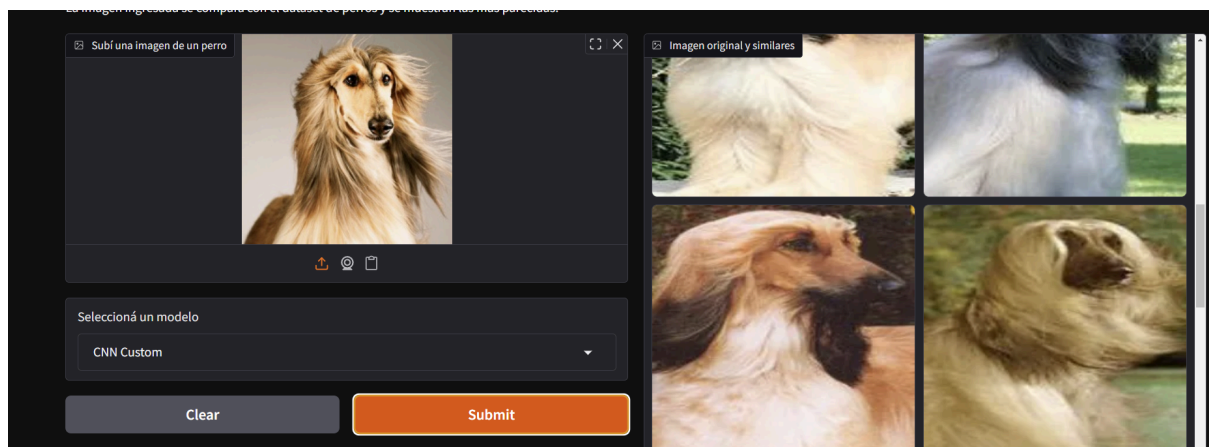
profundidad en la etapa de extracción de características pasando de 256 a 512 y posteriormente a 1024.

Algo que también tuve complicaciones y no tiempo a solucionar, es que el modelo custom tiene dificultades a la hora de realizar embeddings para la búsqueda de similitud, seguramente agregando una capa intermedia para entrenar la creación de los mismos. separada de la clasificación.

Actualización Gradio

Se actualiza la interfaz de Gradio para que ahora el usuario pueda elegir entre un modelo u otro (Fine-Tuning o Custom CNN).





ETAPA 3

Resumen

En esta etapa se nos propone utilizar un modelo YOLO Pre-entrenado para detectar la presencia de perros en una imagen compleja de entrada, una vez detectado hacemos un crop sobre los perros encontrados y cada recorte clasificarlo con el mejor modelo de la etapa anterior. Como último paso actualizar la interfaz Gradio para poder visualizar todo lo mencionado.

resolución

Lo primero en esta etapa es elegir que modelo de YOLO elegir, para esto fui directamente a la documentación de ultralytics, en la misma leí un poco de toda las versiones de YOLO, empezando por la yolov8n que se nos da de ejemplo hasta la última versión que está publicada en la página(yolo12). Si bien de cada versión hay información muy relevante como el mAPaval, la velocidad, cantidad de parámetros, etc.

Decidí ser más rustico y probar todos los modelos desde el yolov8n hasta el yolo12 y ver cual me detectaba mejor los perros de un mini dataset de 22 imagenes buscadas en internet.

```
from ultralytics import YOLO
yolo_8 = YOLO("yolov8n.pt")
yolo_9 = YOLO("yolov9c.pt")
yolo_10 = YOLO("yolov10n.pt")
yolo_11 = YOLO("yolo11n.pt")
yolo_12 = YOLO("yolo12n.pt")
```

✓ 2.0s

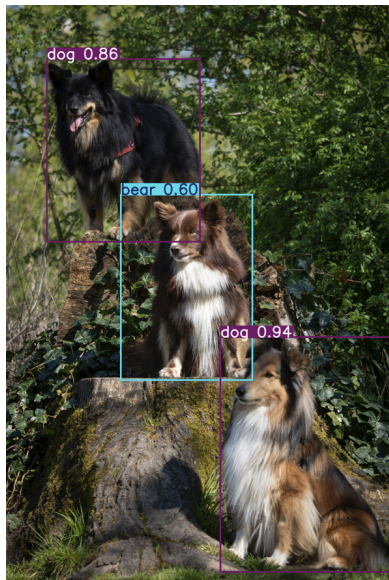
encontrado_yolo8	encontrado_yolo9	encontrado_yolo10	encontrado_yolo11	encontrado_yolo12
✓ 0.0s	✓ 0.0s	✓ 0.0s	✓ 0.0s	✓ 0.0s
[(1, 0), (2, 1), (3, 1), (4, 5), (5, 2), (6, 2), (7, 4), (8, 2), (9, 6), (10, 4), (11, 3), (12, 2), (13, 3), (14, 2), (15, 3), (16, 1), (17, 5), (18, 2), (19, 3), (20, 0), (21, 3), (22, 3)]	[(1, 2), (2, 5), (3, 1), (4, 5), (5, 2), (6, 3), (7, 3), (8, 2), (9, 5), (10, 4), (11, 4), (12, 2), (13, 4), (14, 3), (15, 3), (16, 4), (17, 3), (18, 2), (19, 5), (20, 1), (21, 3), (22, 3)]	[(1, 0), (2, 4), (3, 1), (4, 5), (5, 3), (6, 1), (7, 3), (8, 2), (9, 4), (10, 4), (11, 2), (12, 0), (13, 3), (14, 2), (15, 3), (16, 1), (17, 4), (18, 2), (19, 4), (20, 0), (21, 3), (22, 3)]	[(1, 0), (2, 1), (3, 1), (4, 4), (5, 2), (6, 3), (7, 3), (8, 3), (9, 5), (10, 4), (11, 3), (12, 3), (13, 3), (14, 4), (15, 3), (16, 1), (17, 3), (18, 2), (19, 6), (20, 0), (21, 2), (22, 3)]	[(1, 1), (2, 2), (3, 1), (4, 4), (5, 2), (6, 3), (7, 3), (8, 2), (9, 4), (10, 4), (11, 4), (12, 3), (13, 3), (14, 2), (15, 3), (16, 1), (17, 3), (18, 2), (19, 5), (20, 1), (21, 2), (22, 3)]

Esta me pareció la mejor manera de mostrar los resultados de cada modelo, lo que muestro es una lista con tupla, donde cada tupla tiene como primer valor el número de imagen y el segundo valor es la cantidad de perros encontrados en la misma por el modelo de YOLO.

la cantidad de perros en el dataset es de 68, a continuacion vemos cuantos detecta cada modelo para el mismo dataset.

yolo8 = 57
yolo9 = 69
yolo10 = 54
yolo11 = 59
yolo12 = 58

Algunas consideraciones, hay veces que los perros alguna versión de yolo no los detecta bien, ya que los confunde con otros animales, por ejemplo la siguiente imagen donde se detecta un perro como oso. También pasa en algunas ocasiones que si un perro está “cortado” por un objeto se detecta 2 veces como perros distintos.



Bien, como mencioné antes, elegí el modelo que más perros de mi mini dataset de 22 imágenes encontrase, esto no se debió a solo mirar dicha cantidad, sino que analicé todas las imágenes con cada uno de los modelos, y más óptimo me pareció ser el que más perros detectaba. Otra opción muy óptima y en ciertos casos mejor, es yolo11.

Ahora que ya elegí el modelo queda armar un pipeline, donde se pasa una imagen a yolo, la misma es recortada cuando se encuentra un perro y con nuestro mejor modelo se clasifica.

Realicé una función que recibe una imagen en formato PII como entrada y retorna la imagen de entrada con los bounding boxes de los perros detectados y una etiqueta de clase.

Cada imagen de entrada es procesada con yolo, se filtran los por los perros que detecta, y de cada perro detectado se extraen las coordenadas x1, y1, x2, y2 de sus bounding boxes, con estas coordenadas se realiza un crop de la imagen y cada recorte es pasado al modelo clasificador, en este caso el fine-tuning del cual obtenemos una raza predicha. Luego de esto se dibuja en una copia de la imagen original, con las coordenadas que ya teníamos los bounding boxes de cada perro encontrado y su raza predicha.

Complicaciones

En esta etapa no hubo complicaciones significativas, fue todo bastante sencillo, lo único que resalto fue la dificultad a la hora de marcar los bounding boxes y razas de los perros, ya que yolo no deja “editar” sus datos, solo filtrar por los mismos, entonces hubo cierta complicación para conseguir dibujar los bounding boxes bien. Y a la hora de poner la raza de cada canino tuve el problema que la raza no se llegaba a apreciar en algunas imágenes o simplemente eran enormes en otras, así que tuve que solicitar ayuda a una inteligencia

artificial para poder setear el tamaño de las etiquetas de las razas para cada imagen, sin importar su tamaño.

Gradio

La interfaz en esta ocasión es muy simple, se utiliza de entrada una imagen en formato PIL que se pasa al pipeline desarrollado previamente y se retorna la misma imagen con los bounding boxes de los perros y sus razas.



ETAPA 4

Resumen

En esta etapa es donde se evalúa el pipeline anteriormente desarrollado. Se solicita crear un dataset de 10 imágenes complejas, crear anotaciones manuales de estas imágenes (creando los bounding boxes para los perros y sus razas correctas). Luego ejecutar el pipeline de la etapa anterior sobre este conjunto de imágenes y medir su desempeño.

También hay que aplicar técnicas de optimización a nuestro mejor modelo de clasificación y detección para acelerar la inferencia. y como última instancia Crear un script que utilice el pipeline completo, para genera anotaciones (como las hechas a mano) pero de manera automática.

Resolución

Para la recopilación de imágenes se entró a diversos sitios y se descargaron más de 80 imágenes, las cuales fueron filtradas y quedaron 22, de las cuales se descartó la primera imagen. Se buscó por imágenes complejas, que haya varios perros por imagen y otros objetos, pero la principal dificultad es poder distinguir razas o encontrar imágenes con estas características y perros de raza en la misma.

Para la anotación manual se utilizaron diversas páginas e inteligencias artificiales, dando como resultado la más óptima makesense.ai, en la misma se pueden cargar todas las imágenes, hacer los bounding boxes a mano de cada perro y ponerle un label con su raza. Esto devuelve un .csv con las siguientes columnas {label_name, bbox_x, bbox_y, bbox_width, bbox_height, image_name, image_width, image_height}

Métricas

Aca primero quiero aclarar que por no entender bien el enunciado saqué las siguientes métricas:

Para el modelo Fine-Tuning se saca (Precisión,Recall y F1-Score).

Para YOLO se saca (IoU,Precisión,Recall y F1-Score).

Luego se calcula para YOLO + Fine-Tuning (Precisión,Recall y F1-Score).

Y como último se calcula el mAP solo para los bounding boxes y el mAP para bounding boxes + raza.

FINE-TUNING

Empezando por analizar las métricas del desempeño del modelo Fine-Tuned con el conjunto de imágenes complejas.

	precision	recall	f1-score	support
Basset	0.00	0.00	0.00	1
Beagle	0.00	0.00	0.00	2
Blenheim	0.00	0.00	0.00	1
Bluetick	0.00	0.00	0.00	1
Border Collie	0.00	0.00	0.00	7
Borzoi	0.00	0.00	0.00	1
Boxer	0.00	0.00	0.00	1
Chihuahua	1.00	1.00	1.00	1
Collie	0.00	0.00	0.00	1
Corgi	0.00	0.00	0.00	5
Dingo	0.00	0.00	0.00	3
Elk Hound	0.00	0.00	0.00	1
French Bulldog	1.00	0.50	0.67	2
Golden Retriever	1.00	0.25	0.40	4
Great Dane	0.33	1.00	0.50	1
Great Perenees	1.00	1.00	1.00	1
Groenendael	0.25	0.50	0.33	2
Labradoodle	0.50	0.33	0.40	3
Labrador	0.50	1.00	0.67	1
None	0.00	0.00	0.00	5
Pit Bull	0.00	0.00	0.00	1

Yorkie	0.00	0.00	0.00	6
micro avg	0.33	0.22	0.27	68
macro avg	0.28	0.26	0.24	68
weighted avg	0.29	0.22	0.22	68

Métricas Valore

Micro avg F1 0.27

Macro avg F1 0.24

Weighted avg F1 0.22

Micro Recall 22%

Micro Precisión 33%

Micro promedio es cada predicción individual por igual (independiente de la clase), refleja el rendimiento general en todo el “dataset”

Macro Promedio promedia los F1 por clase sin importar cuántas imágenes tenga cada una. Un valor de **0.24** muestra que muchas clases tuvieron un rendimiento pobre (varias con 0 en todas las métricas).

Weighted promedio, promedia según cuántas imágenes tiene cada clase. Es útil cuando algunas clases tienen muchas más muestras que otras. Un F1 de **0.22** muestra que el mal rendimiento está presente en varias clases con más peso también.

Viendo esto puedo decir que el modelo funciona bien con imágenes limpias, pero flaquea o sufre en escenarios “reales”, esto se puede ver en el bajón de métricas con respecto a las mismas con el dataset de test evaluado en la etapa 2.

Las imágenes con poco support (muestras) no lo tomaría como valores confiables para evaluar, pero igualmente tienen peso y podrían marcar una tendencia.

Algunas clases tienen un Recall alto, pero precisión baja como el siberian husky, donde el modelo encuentra varios casos verdaderos, pero también se equivoca bastante cuando predice esta clase.

Los casos que llaman la atención son los que tienen 0.0 en recall ya que el modelo nunca predijo correctamente que una imagen era de esa raza y 0.0 en precisión quiere decir que tampoco por error predijo esa raza en otra imagen.

La conclusión de estas métricas sugieren que aunque el modelo fine-tuning tiene un buen rendimiento en condiciones controladas, le cuesta generalizar en imágenes desafiantes, esto puede ocurrir porque cuando se entrena con el dataset de train las imágenes estén “similares” y el modelo aprende muy bien ese contexto, pero cuando se le muestra algo distinto empieza a tener dificultades.

Para solucionar esto se me ocurren 2 cosas principalmente, la primera y la cual no sé qué tanto impacto tendrá, es usar data augmentation más agresivo. La segunda y la que veo más viable es entrenar con un dataset más diverso y complejo desde un inicio.

YOLO DETECCIÓN

Ahora vamos a hablar de las métricas solo de yolo en detección de bounding boxes.

```
[YOLO] Precision: 0.8806, Recall: 0.9365, F1: 0.9077
```

```
IoU promedio (detección de bounding boxes): 0.7822
```

Métricas	Valore
Precisión	0.8806
Recall	0.9365
F1-Score	0.9077
IoU promedio	0.7822

En la precisión vemos que los bounding boxes que el modelo predijo como perro, el 88% son correctos y que en una menor medida comete falsos positivo, dibujando bounding boxes donde no hay perros.

Si hablamos de recall podemos decir que el modelo detecto el 93% de los perros que realmente hay en “dataset”

Y con el F1-score corroboramos que hay un equilibrio entre precisión y recall, teniendo un valor de 0.90 confirma que el modelo no solo encuentra bien los perros, sino que también acierta en la mayoría de predicciones.

IoU (intersection over union) mide que tan bien se ajustan los bounding boxes predichos con los reales, se calcula como el área de superposición entre ambas, dividido por el área total unida.

Teniendo en cuenta esto un IoU de 1.0 sería una superposición perfecta. y mientras más bajo el valor menos preciso. En este caso obtuve un promedio de 0.78, que indica que los

bounding boxes hechos por yolo se ubican bastantes similares a los ajustados a mano por mi.

Luego de evaluar todo esto podemos decir que yolo tiene un desempeño sólido, tiene un valor alto de Recall, una buena precisión y el IoU promedio cercano a 0.8. Básicamente encuentra bien los perros y sus bounding boxes

YOLO + FINE-TUNING

Ahora voy a evaluar cuántos perros fueron correctamente detectados y clasificados con la raza correcta al mismo tiempo, en el dataset de imágenes complejas. Esta es una evaluación mucho más exigente que analizar detección o clasificación por separado.

```
[YOLO + Fine-Tuning] Precision: 0.2239, Recall: 0.7895, F1: 0.3488
```

Métricas	Valore
Precisión	0.2239
Recall	0.7895
F1-Score	0.3488

Podemos observar el valor de la precisión bajo, queriendo decir que el 22% de la predicciones fueron completamente correctas (bbox + raza). El sistema hace muchas predicciones incorrectas, ya sea porque detecta bien la ubicación, pero se equivoca con la raza o porque detecta un bounding box que ni siquiera corresponder a un perro real (falso positivo)

Con un recall de casi 79% el pipeline logra detectar correctamente la ubicación y raza de los perros reales del conjunto de imágenes.

F1-Score como ya dijimos es el equilibrio o promedio entre precision y recall ($2(\text{precisión} * \text{recall}) / (\text{precisión} + \text{recall})$), aunque el pipeline encuentra muchos perros, el bajo nivel de aciertos totales (precisión) tira muy abajo el resultado global. Esto nos muestra que el modelo necesita mejorar en precisión, especialmente en la clasificación de razas

mAP evalúa la calidad del sistema completo de detección y clasificación, pero considerando el rendimiento por raza. A diferencia de las métricas globales como precisión o recall del pipeline completo, el mAP analiza cada clase por separado y luego promedia los resultados, el profe Manson dijo que es muy conveniente cuando hay muchas clases, ya que muestra si el sistema funciona consistentemente o solo acierta unas pocas.

el mAP obtenido fue de 0.1870 esto significa que en promedio, el sistema logra en promedio un 18% del área debajo de la curva precisión-recall para cada raza (del conjunto evaluado)

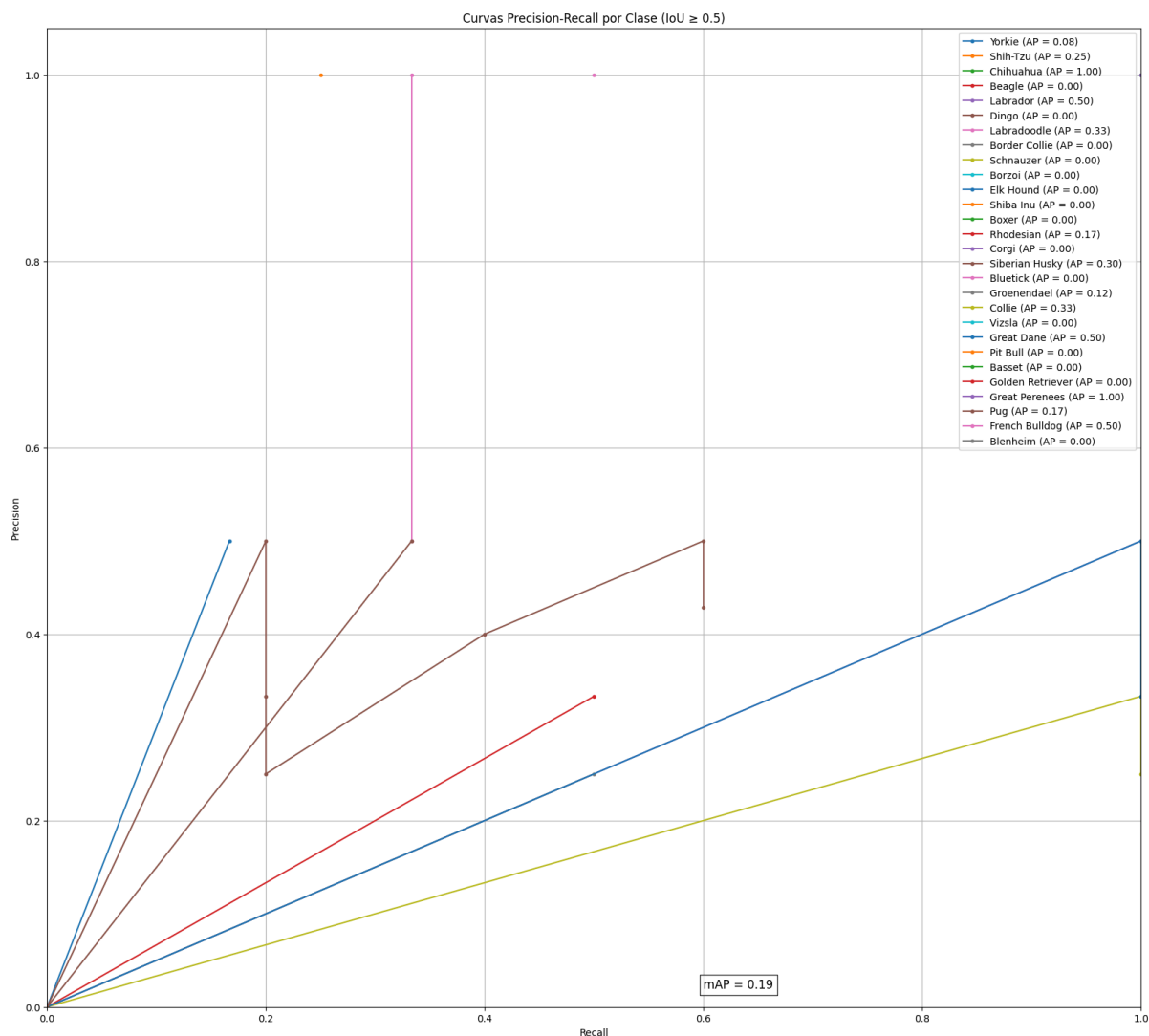
Para calcular esto fue todo un dolor de cabeza, lo voy a admitir, tuve que solicitar ayuda a diversas inteligencias artificiales, voy a hacer un desglose de cómo se calcula.

Para cada raza (clase), se agrupan todas las predicciones y ground truth. Se comparan los bounding boxes predichos y los reales usando IoU (≥ 0.5). Se clasifican como TP o FP, dependiendo de si acertaron o no, se genera una curva Precisión-Recall por clase, graficando como cambia la precisión a medida que se aumenta el recall.

Se calcula el AP (average precisión) como el área bajo esa curva para cada raza y finalmente, se calcula el mAP promediando los AP de las clases presentes.

Considero que era esperable que esta métrica de bajo, luego de ver los problemas que tiene mi modelo de fine-tuning para clasificar este nuevo conjunto de imágenes, ya que se requiere clasificar bien la raza de cada perro y bien su bounding box, a su vez, cualquier error en cualquiera de estos cuenta como un fallo (fp). Este valor nos dice que, en promedio, la confianza del sistema baja a medida que se intenta recuperar más instancias correctas.

De esta métrica podemos deducir lo que mencioné anteriormente y que el cuello de botella es el clasificador del pipeline, ya que yolo detecta bien los perros. Las curvas de precisión - Recall individuales pueden ayudar a identificar qué razas se predicen bien y cuáles no. Básicamente el modelo está teniendo problemas para generalizar las imágenes, quizá se deba a ángulos, iluminación, o que simplemente el dataset de entrenamiento era muy similar.



El gráfico muestra las curvas Precision - Recall para cada raza presente en el dataset de imágenes complejas, considerando un umbral de IoU ≥ 0.5 . Cada línea representa una clase y su desempeño específico en cuanto a detección y clasificación correctas.

podemos ver clases con un excelente desempeño como Chihuahua (AP = 1.00) y Great Perenees (AP = 1.00), un desempeño más normal, French Bulldog (AP = 0.50) y Labrador (AP = 0.50) y ya clases con un pésimo desempeño Shih-Tzu (AP = 0.225), Siberian Husky (AP = 0.30), Rhodesian (AP = 0.17) o peor.

El gráfico confirma que el rendimiento del sistema varía mucho entre razas. Algunas razas se detectan y clasifican bien (aunque pocas), pero la mayoría presenta curvas planas o poco desarrolladas, lo cual refleja errores frecuentes o falta de confianza. La gran dispersión entre curvas, y la presencia de muchas clases con AP = 0, explican el bajo mAP general (0.19).

Optimización

En esta parte se intentó probar dos métodos de optimización con el objetivo de reducir los tiempos de inferencia o mejorar la portabilidad del modelo, sin perder precisión. Las técnicas aplicadas fueron Cuantización dinámica (pytorch) y exportación a ONX

En la cuantización dinámica se convierten las capas lineales (nn.Linear) a una versión más liviana, usando entero de 8 bits (int8) en lugar de flotantes de 32 bits. Esto con la esperanza de que la velocidad de inferencia sea menor.

En ONNX se exporta el modelo a este formato, lo que permite usarlo fuera de pytorch (en motores como onnx runtime o TensorRT). se usa un dummy_input de tamaño 224x224 y se configuraron los ejes dinámicos para permitir batches de diferentes tamaños. al hacer esto se espera portabilidad a otros entornos y una mayor inferencia.

```
ONNX Simplificado -> Acc: 0.9500, Tiempo medio: 2576.05 ms
Cuantizado -> Acc: 0.9500, Tiempo promedio: 3215.43 ms
Original PyTorch -> Acc: 0.9500, Tiempo promedio: 3219.36 ms
```

Luego de testear los modelos podemos observar que la optimización dinámica no tuvo una mejora real en velocidad, el tiempo promedio fue 3215 y la del modelo original fue 3219. Ambas tuvieron el mismo ACC.

Por otro lado el modelo ONNX simplificado mantiene el mismo ACC que el modelo original, pero con una inferencia de 2576ms contra los 3219 del modelo original.

La cuantización dinámica no aportó mejoras significativas a este caso, probablemente porque no se cuantifican capas convolucionales, solo lineales.

El modelo exportado a ONNX sí mostró mejoras de velocidad manteniendo precisión, por lo que es una opción válida si se desea usar el sistema en entornos externos a pytorch.

Anotación automática

De esta parte la verdad no tengo que decir mucho, la verdad estaba todo muy bien documentado y fue simplemente investigar por los formatos de yolov5 y coco para los returns.

CONCLUSIÓN

Para cerrar este informe, quería comentar un poco mi experiencia general durante el trabajo práctico.

Hubo partes desafiantes, otras muy entretenidas y muchas donde aprendí realmente cosas nuevas. La primera etapa fue bastante clara, me ayudó a entender bien cómo usar modelos pre-entrenados y trabajar con embeddings, y no tuve mayores complicaciones.

La segunda etapa fue la que más disfruté (y también la que más tiempo me llevó). Al principio me costó arrancar, no sabía muy bien cómo abordar el entrenamiento de modelos, pero una vez que entendí lo que se pedía, me enganché con la idea de mejorar los modelos cada vez un poco más. Fue donde más aprendí sobre fine-tuning y arquitectura de redes.

La etapa tres fue más sencilla: consistió básicamente en integrar YOLO para detectar perros, y funcionó bien.

La cuarta etapa fue la más frustrante, sobre todo por tener que generar manualmente los *bounding boxes* cuando las herramientas automáticas fallaban. Además, al probar el pipeline completo, las primeras métricas eran muy bajas y eso fue un bajón. Recién después, entendiendo bien cómo se evaluaba cada parte (y sobre todo cómo se calculan las métricas), pude interpretar mejor los resultados.

En cuanto a la posibilidad de aplicar este sistema en un caso real, hoy por hoy no lo veo preparado para eso, principalmente por la baja precisión al clasificar razas en imágenes reales. Sin embargo, con mejoras específicas —como entrenar con imágenes más variadas, mejorar el clasificador de razas, o refinar los *bounding boxes*— el sistema podría tener potencial real.

Más allá de eso, me voy muy conforme con todo lo que aprendí, sobre todo en temas como métricas, evaluación, entrenamiento de modelos y visión por computadora en general. No solo entendí conceptos que antes me costaban, sino que pude ponerlos en práctica y ver cómo afectan al rendimiento final.